

HSCA Final Project Report

Ryan Kerns

December 2025

1 Introduction

The goal of this project was to design and implement a half-precision (binary16) FMA that computes:

$$R = X \cdot Y + Z$$

by IEEE 754 floating-point standards. It should:

- Takes three 16-bit inputs: x, y, z
- Optionally multiplies x*y and then optionally adds/subtracts z, to implement both a plain add/mul path for the test vectors
- Supports 4 rounding modes (RZ, RNE, RP, RN)
- Handles all the special cases (zeros, infinities, NaNs)
- Returns a 16-bit result plus 4 exception flags INV, OF, UF, NX

At a high level, the module flows like:

1. Decode the 2-bit roundmode as 4 one-hot signals
2. Each input is split into sign, exponent, and mantissa
3. From these fields we classify each operand as:
 - Zero: exp=0; frac=0
 - Inf: exp=all ones; frac=0
 - NaN: exp=all ones; frac not 0
4. We then handle the product P as either X*Y for the mul path, or just X for the pure add path. We compute the effective signs of P and Z, including the negr/negz controls.
5. Multiply Core:
 - When the operation involves multiplication (mul=1), the module computes $x * y$ in normalized form.

- It builds the 11-bit significands with an implicit 1 for normal numbers.
- Compute the raw exponent sum: $\text{exptemp} = X_e + Y_e - \text{bias}$ (bias = 15, extended to signed 7 bits).
- Multiply: $P_m = X_{\text{sig}} * Y_{\text{sig}}$ (22-bit product). Depending on the top bit of P_m : If $P_m[21]$ is set, the product is in $[2,4)$, so it is normalized by shifting right one and incrementing the exponent. Otherwise, it is in $[1,2)$, and the exponent is left as exptemp .

6. Add Core:

- When $\text{add}=1$, the unit enters the general add/FMA datapath. This same block is used for: Fused FMA: $\text{mul}=1, \text{add}=1; (+/- X*Y) +/- Z$. Pure add: $\text{mul}=0, \text{add}=1, (+/- X) +/- Z$.
- Effective signs include optional negation: $\text{signX}_{\text{eff}} = X_{\text{sign}} \hat{\text{negr}}$ and $\text{signZ}_{\text{eff}} = Z_{\text{sign}} \hat{\text{negz}}$
- Z's significand is extended similarly (implicit 1 for normal exponent; 0 for zero/subnormal), with three low zeros.

- The operand with the larger exponent is chosen as A; the other becomes B.
- We then determine if $\text{signA}_{\text{add}} == \text{signB}_{\text{add}}$. If so then we use the same sign addition path, if not, then we use the subtraction path
- After special cases and arithmetic cores are computed, the module combines everything and produces the final result and flags
- result is chosen from special_result when special, otherwise from the overflow/ underflow-adjusted p_{finite} .

2 Test Results

Lint Results

As output by the terminal:

```
a20275139@avatar:/final-project-high-speed-computer-
arithmetic-fall-2025-rykerns/fma16$ ./lint-fma
```

```
- V e r i l a t i o n   R e p o r t: Verilator 5.027 devel rev
v5.026-55-g095b1ccb6
- Verilator: Built from 0.033 MB sources in 2 modules,
into 0.055 MB in 5 C++ files needing 0.000 MB
- Verilator: Walltime 0.011 s (elab=0.001, cvt=0.007, bld=0.000);
cpu 0.000 s on 1 threads; allocated 20.840 MB
```

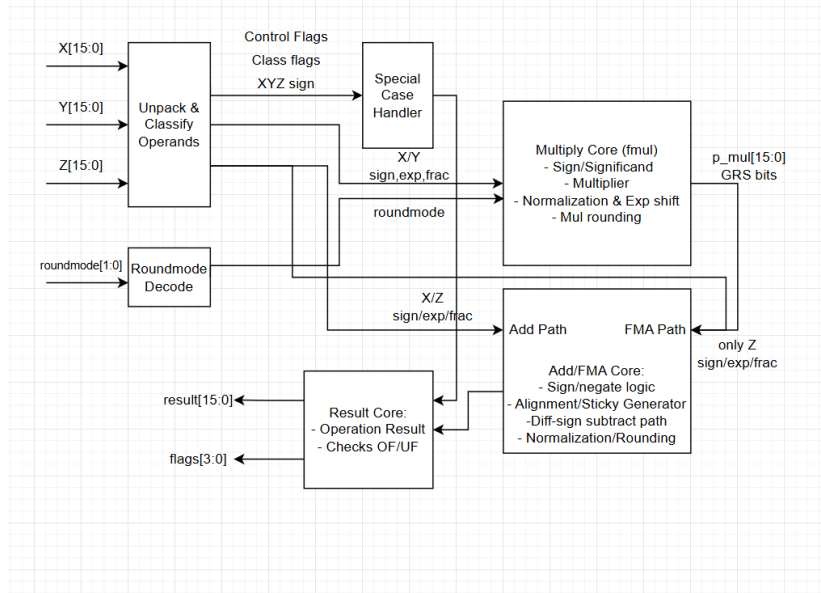


Figure 1: High-Level Block Diagram

fma16 passed lint

Although the `fma16` module passes many of the vectors its given, there are still a bunch of corner cases where the vectors still fail. Most of the remaining failures are off-by-one errors, where the target vector is ± 1 ULP away from the computed vector, these errors looked something like this:

```
Error: inputs b293 * bc01 + 13fe
#  result = 329c (329d expected) flags = 0001 (0001 expected)
```

Table 1 holds the best-case tests I was able to get from each `.tv` file. Initially the number of errors for each were much higher, but through some trial-and-error solutions I was able to try narrowing the range of errors for each test, I committed major benchmarks to the GitHub Classroom.

The original rounding block was implemented so that the inexact bit was always rounded up if there was a roundmode flag. Which does work for RZ only if it's overridden, but IEEE-754 requires nearest ties to even, not nearest ties away from zero. RP and RM signs were also not handled correctly. I added the guard/round/sticky bits at the bottom of the normalized significand, for RNE, we round up only if guard is 1 and (round or sticky or LSB) are 1. This got rid of most of the big rounding errors in `fadd` and `fsub`.

Many early failures of the fma were where both the exponent and mantissa went the wrong way. I reworked the normalization after the product+add to try and reduce some of the errors in the fma tests. Added detection for leading zeros

Name of Test	Pass/Fail (No. of Errors)
fmul_0	Pass (0)
fmul_1	Pass (0)
fmul_2	Pass (0)
fadd_0	Pass (0)
fadd_1	Pass (0)
fadd_2	Pass (0)
fma_0	Pass (0)
fma_1	Fail (3)
fma_2	Fail (124)
fma_special_rz	Fail (28)
fma_special_rne	Fail (28)
fma_special_rp	Fail (20)
fma_special_rm	Fail (100)
baby_torture	Fail (189)

Table 1: Summary of test results

so that the exponent would shift left and right by 1 where needed. The errors that remained were same sign, same exponent, and the mantissa being off or it had an NX flag mismatch.

In the remaining errors that I couldn't get rid of, the sign and the exponent almost always match the golden result with the mantissa being 1 ULP larger than expected. Many of these cases are flagged as inexact, and the implementation usually classes it as either a) round up when it should have rounded to even, or b) round up and clear NX so we claim it as "exact." I think this issue is caused by too aggressive rounding when we compute X and Y . Since NX is set if any of the discarded bits are non-zero, but rounding up are far more selective. `round_up = G && (R || S || U[0]);` is the rounding function, if the remainder is strictly less than a half-ULP, then don't round up, but NX may still be 1 (if R or S are set).

If the remainder is exactly a half-ULP: G=1, R=0, S=0: then if $U[0] = 0$ (even), don't round up; NX = 1, but value stays. If $U[0] = 1$ (odd), round up to make the result even. The fact that exponents are correct, fadd/fmul mostly pass, but these FMA cases are all 1 ULP high, strongly suggests our remaining rounding-up and NX are not fully implemented per IEEE-754.

3 Synthesis Results

On the avatar server I was not able to run the SKY90 Synthesis for the FMA16 design, `build_me` and `clean_me` seemed to work, but when running `make synth`, the terminal would output:

```
DC Synthesis
```

```
dc_shell-xg-t -64bit -f scripts/synth.tcl | tee runs/10000
_MHz_2025-12-09-22-29/synth.out
/bin/sh: 1: dc_shell-xg-t: not found
```

4 Project Management

Project management on this assignment was a mixed success, largely because I started the project very late in the semester. Instead of having time to follow a structured sequence, most of the work collapsed into the last part of the schedule. That meant the design, debugging, and documentation all overlapped in the final days, and I had to prioritize getting a functionally correct FMA core and passing as many tests as possible over deeper optimization.

Even with the compressed timeline, I tried to keep some basic project management discipline. I used the GitHub Classroom repository to track versions through commits, and re-ran all tests when I made a major change to the design to make sure that the new stuff didn't break any existing modules.

The main takeaway from this project is that for a design of this complexity, starting earlier is essential, as I've spent a significant amount of time crunching to get this project done, whereas doing little bits every week would have reduced stress and last-minute debugging. A more intentional schedule would have reduced debugging/errors and allow me to deliver a fully correct FMA and complete backend results.

5 Conclusion

The final module supports multiplication, addition, and a fully fused FMA operation, along with all four required rounding modes and basic handling of IEEE special values such as zeros, infinities, and NaNs. A small but persistent set of off-by-one errors remains in some of the more difficult rounding cases, especially where the fused operation interacts with guard/round/sticky bits and underflow. In addition I was unable to complete the synthesis step, so I am unable to report the area, timing, or PPA.